Speaker scripts, spoken version — progress_2026-11-11_v8.pptx (all 61 slides)

Written to be said aloud: short sentences, plain words, each slide leads into the next. Numbers and claims match what is on each slide. Slides 1–37 are the main talk (about 35 minutes at a relaxed pace); 38–61 are references, backup and glossary.

1 H2S-NN: Heterogeneous Heterophilic Sheaf Neural Network with Path Instances

Good afternoon, everyone, and thanks for coming. Today I'll walk you through H2S-NN, the model we're building for graphs that have many types of nodes and edges, and where connected nodes often don't share a label. I'll cover the problem, the idea, the method, and how we plan to test it. One thing up front: the model is finished and tested, but the big experiments haven't run yet. So there are no accuracy numbers today, and I'd rather tell you that now than have you wait for them.

2 Reviewer feedback on the submitted version

Let's start with where the paper stands. We sent the earlier version to WSDM and it came back split. Two reviewers leaned accept, two leaned reject, and the meta-reviewer recommended reject. What I find encouraging is what they agreed on. All of them liked the core idea: keep each path instance as its own object, instead of squashing it into a link between its two ends. Three found the construction principled, and they liked the range of tasks. So the idea survived. The criticism was about evidence, and that's the next slide.

3 Reviewer feedback: the five main concerns

These are the five concerns, shortened but in the reviewers' own terms. One: we never showed heterophily. Only IMDB was strongly heterophilic. Two: the face only reached the nodes through a closing edge, so it wasn't really isolated. Three: no scalability study, and our ACM run used 9.3 gigabytes and three seconds an epoch. Four: it was unclear how baselines were tuned, and we quoted HetSheaf instead of running it. Five: no failing example to motivate the design. The first three hit the core claim. The last two are about how we tested and explained it. This whole talk is our answer to these five.

4 Update since last meeting

Here's the quick before and after; I'll come back to it at the end. The paper used to have three contributions. Now it has one core idea plus a sampler. The face used to reach the nodes through a closing edge. Now there's one energy and one solve. Path selection used to be a schema search with budgets. Now it's random walks, with a learned weight for every path. And the evaluation grows from eleven datasets to twenty, with every baseline rerun under one protocol. Status: implemented and tested, ACM memory down from 9.3 to 2.8 gigabytes, and no accuracy runs yet.

5 Outline

Here's the plan, in five parts. Background first, so we share the vocabulary. Then the problem and our research questions. Then the method, which is the longest part. Then the experiments we plan to run. And last, a short discussion of what's new and what's still open.

6 A heterogeneous graph: nodes/edges have types

Let me set up a small example that I'll use all the way through. This is a citation graph with three kinds of nodes: papers, authors and subjects. P1, P4 and P9 are papers, A7 is an author, S3 is a subject. The edges have types too: writes, has-subject, cites. Say we want to classify papers, math or biology. The point of this slide is that types carry meaning. An author neighbour tells you something different from a subject neighbour, and a model that ignores types throws that away.

7 Meta-paths: a type pattern versus one concrete path

A meta-path is a type pattern. Take paper, author, paper, PAP for short. It reads as 'two papers share an author'. That's the schema. An instance is one real path that follows the pattern. P1, A7, P9 is one. P1, A7, P4 is another. Notice that A7 sits in the middle of both. We call it the intermediate node, and it's going to matter a lot later. Meta-paths are the standard tool for typed multi-hop relations; APA, for example, means co-authors.

8 Heterophily: connected nodes have different labels

Now the problem. In a homophilic graph, linked nodes tend to share labels, so averaging your neighbours keeps the label. That's why GCN works there. Heterophily is the opposite. P1 and P9 share the author A7, but they work in different fields. Average them and the labels blur. In heterogeneous graphs the

same thing happens along typed paths. Being connected doesn't mean being similar, and plain averaging can actually hurt.

9 Measuring heterophily: H2[6] on heterogeneous graph

How do we measure this on a heterogeneous graph? We use H-squared, from the H2GB benchmark. Take the length-two meta-paths, and for each one build the paper graph it induces, so PAP links papers that share an author. Then measure adjusted homophily on that graph. 'Adjusted' means it's corrected for class imbalance, so one big class can't make a graph look homophilic. That correction is why I prefer it over the plain label measure. When we get to the datasets I'll show it in heterophily form, where one means every linked pair has different labels.

10 Sheaf diffusion: agree inside the edge, stay different

This is the key tool, in a picture. Two nodes, u and v, joined by an edge e. In a GCN you pull x_u and x_v towards each other, so the gap is just their difference, and the two merge. A sheaf adds one learned map for each end. F_u takes u into the space of the edge, F_v does the same for v, and the gap we penalise is between the mapped vectors, inside e. One step moves both nodes to close that gap. Here's a concrete case. F_u rotates by plus thirty degrees, F_v by minus sixty. After convergence, u points east and v points north. They are still different, but inside e they point the same way. So diffusion only removes the disagreement the maps can't explain, and each node keeps its class.

11 How heterophily is handled: decompose, then fuse

So how do existing methods deal with it? Mostly by decomposing and then fusing. Step one, the original graph. Step two, build a separate graph for every schema. The PAP view links papers through authors, the PSP view through subjects. Look at what disappeared: A7 and S3 are gone, only papers are left. Step three, run a GNN on each view; this is where heterophily gets handled. Step four, fuse the views with attention and predict. So heterophily is studied indirectly, on derived graphs, one per schema.

12 Two limitations of decompose-then-fuse

That has two limitations. First, heterophily is judged on derived graphs. You get one homophily number per view, the original typed edges are never used, and the whole thing depends on a hand-made schema list. Our goal here is to handle heterophily on the original graph, in one model. Second, pairs differ only by their endpoints. P1, A7, P9 and P1, A2, P9 both collapse into the same P1–P9 edge with

weight two. Only a count survives, and A7 never learns that P1 and P9 are linked. Our goal: let path instances act as structure. Both limitations come from the same step, replacing the graph with schema views before any learning starts.

13 Our view and the research questions

So here's our view. Stay on the original graph, no per-schema views. Treat meta-paths as structure rather than input: each path instance becomes a face, attached to its nodes and edges, with no closing edge between the ends. And give each instance a learned weight that depends on its nodes and relations. Back to the running example: does it matter who A7 is? Should A7 learn that P1 and P9 are linked, and part of a meta-path? That gives us two research questions. RQ1: can a single model on the original graph handle heterophily using path instances, with no per-schema branches? RQ2: do path instances, used as structure, carry information that schema-level encoders and instance features don't?

14 Related work: where each family puts the path

Where do existing families put the path? Decompose-then-fuse methods like Hetero-squared-Net don't work on the original graph and don't use the instance. Schema-level encoders, HAN and SeHGNN, are the same, and don't target heterophily at all. Instance encoders like MAGNN do use the instance, but as a feature, and only the target node gets updated. Sheaf networks, NSD and HetSheaf, stay on the original graph with learned edge maps, but only edges. Cell and hypergraph networks do put higher-order cells on the graph, with free maps, and still don't target heterophily. Ours is the only row that stays on the original graph and lets one path update all of its nodes.

15 Objectives and contributions

Given that, what must we show, and what do we offer? Two objectives, one per question. O1: one model on the original graph handles heterophily, with no per-schema branch. O2: paths used as structure beat schema encoders and instance features. And three contributions. First, path instances as faces of the graph's own sheaf, with maps composed from edge maps, reaching every node, no closing edge. Second, one energy and one solve, with a unique answer for any parameters, so there's no depth to tune. Third, WISP, a sampler that needs no schema list, with unbiased random walks and a learned weight per instance. One design choice I want to point out: every comparison our questions need is just a configuration of the same model on the same sampled paths.

16 H2S-NN in one picture

Here's the whole model. I'll spend the next ten slides on it, so just read it left to right for now. The input is the typed graph itself, with no meta-path list. Step two encodes every node, whatever its type, into one shared d-dimensional space; call that vector h. Step three, WISP samples path instances by random walks, and each one becomes a face with a weight. Step four builds the sheaf: learned maps for both ends of every edge, and the face maps are composed from those. Step five is the heart. Node representations are the minimiser of one energy: stay close to h, and agree with your neighbours after the maps. Step six, a small MLP reads each node's own encoding and the propagated one, and predicts for the task, node, link or graph. Training sends the gradient back, the red arrow, through one extra linear solve instead of through unrolled layers.

17 Step 1–2: encode nodes, learn a map for each edge end

Steps one and two. First, a type-specific encoder puts every node into the same space; if a type has no features, it's just a learned vector. Then for each edge, say between P1 and A7, we learn a map for each end: one for P1 into the edge, one for A7 into the edge. A small network reads both ends, the relation and the roles to produce them. The maps rotate or reflect but never shrink, which keeps things stable. They depend on who both ends are, and role frames keep the direction. Each edge also gets a weight, a_e. All of this is computed once per forward pass, and the same maps are used by edges and, composed, by faces.

18 WISP (1): sample path instances by random walks

Where do the path instances come from? WISP samples them with random walks. Look at all the length-two walks from P1. From P1 you go to A7 or S3, half each. From A7 you reach P9 or P4, half each again. Multiply by the chance of choosing length two, and each of those walks has probability one over twelve. Through S3 the only unvisited neighbour is P9, so that route gets one over six. One walk becomes one face, up to length four, and we draw new walks every epoch. There's no schema list and no labels involved. And it's unbiased: with the correction factor, the sampled faces give, on average, the operator of the walk distribution we stated.

19 WISP (2): a learned weight for every instance

Sampling gives us candidates. The weight says how much each one counts. Take two instances of the same schema, PAP: P1, A7, P9 and P1, A2, P9. Same schema, but the weights can be very different, because the weight network sees who is actually on the path, all nodes and relations, in order. It's one small MLP per path

length. If you later want to know which schemas the model relies on, you can read that out by summing the weights per schema. Compare that with the submitted version, which ran a schema search with three budgets. Here there's no schema step at all.

20 Step 3: a path instance becomes a face

Step three, from instance to face. Here's P1, A7, P9. The nodes carry states x, the two edges carry states s, and the face carries a state z. The face sits above its nodes and edges, that's the order on the cells. Now notice what's missing: there is no P1–P9 edge, not even implicitly. The face couples to its nodes and edges directly: node to edge, edge to face, node to face, and we solve for all three kinds of state. So A7 is inside the computation. It's coupled to the face directly, and also through its two edges.

21 Face maps are composed from edge maps

A face needs to compare its nodes, but their vectors live in different spaces. We use the same trick as for edges. The face maps are composed from the edge maps along the path. Take A7. We carry it into e1, then read it in P1's frame, and that's A7's map into the face. We just walk along the path, edge by edge. On this example, P1's frame is the face frame. For the edge-to-face maps, every route through the path gives the same answer. This buys us several things. No new parameters. The map depends on who A7 is. The order of relations matters. And it never contradicts the graph's own sheaf, which free maps would lose. The one-line version: a face compares its nodes after carrying them along its own edges.

22 Step 4: one energy over nodes, edges and faces

Step four is the one energy. It has three kinds of terms, colour coded. Orange: node against edge, after the maps. Green: edge against face. Pink: node against face. Each is a squared disagreement, weighted by a_e from step two or by beta_f from WISP. All the states sit in one stacked vector, Y. The table tells you where each symbol came from: a and beta are the weights, F was learned in step two, and R and T were composed in step three. The state s is where an edge's two ends and its faces have to meet, and z is where all the cells of one path meet. Since every term is a square, E is a quadratic in Y with some matrix L. We normalise it by degree to get E-hat, so hubs don't dominate. And step five minimises E-hat.

23 Every node of the path hears the face

Why does this matter for the reviewers' second concern? In the submitted version, we proved that the face-to-node block of the operator was exactly zero, however many steps you ran. The only connection was the closing edge, which gave a P1–P9 shortcut. Now look at the table. Face to P1, face to A7, face to P9: all direct. And no shortcut at all. All three nodes receive the face in a single application of the operator. So that concern is removed by construction. Whether it actually helps accuracy is something we test later.

24 Step 5: minimise the energy while staying near the encoding

Step five is where the energy gets used. We don't minimise E-hat alone, because that would wash out everything. We add an anchor. The objective J is half the distance between x and h, which says stay near the encoding from step one, plus tau times E-hat, the energy from step four. Y-star is the minimiser. Every symbol comes from an earlier step: Y and E-hat from step four, h from step one, and tau is one learned scalar. Set the gradient to zero and you get one linear system, P-transpose-P plus tau L-hat, times Y-star, equals P-transpose h. We solve it with conjugate gradient in about ten steps, and the answer is unique for any parameters. On the right is the node view. If we eliminate the edge and face states exactly, we get x-star equals the inverse of I plus tau S-hat, applied to h. That's a filter: the smooth part passes, the rest stays in h minus x-star, and the head sees both. So J defines the forward pass, with no layers and no depth. The training loss is on the next slide, and it's a different thing.

25 What is learned, and what the loss is

So what is learned, and what is the loss? The table lists each learned piece and where it enters J. The type encoders give h. The edge-map network gives F. Then the edge weights, the instance weights beta from WISP, one scalar tau, and the head, which acts after the solve. What's not learned: the face maps, since they're composed from F, and the sampler and the solver. The loss is the usual cross-entropy, or binary cross-entropy for IMDB and for links. J is not the loss. The gradient goes back through one adjoint solve with the same matrix, so memory depends on the number of states, not the number of solver steps. This also answers 'where do the maps come from?' The task loss shapes them, so the model learns what agreement means on each graph.

26 How the method answers the research questions

Let me tie the method back to the two questions. For RQ1, the mechanisms are the edge maps, the anchor to h, and the head reading both x-star and h. We test it

by plotting the gain against H-squared over eight datasets, by turning the maps into identity, and by comparing with Hetero-squared-Net. For RQ2, the mechanisms are the face state, the composed maps and the weight beta. We test it by removing faces, replacing the face with a closing edge, or with a feature, and making the weights uniform. For both questions there is WISP, tested against an expert schema list on the same model, and against LMSPS schemas, which is still only a proposal. The rule we followed: each mechanism was added for one requirement, and each has an ablation that can remove it.

27 Heterophily metrics: labels (MLH) and features (MDE)

Now the experiments. First, how we measure heterophily in the datasets. MLH asks: do linked nodes share a label? You take the graph a meta-path induces and check label agreement. Near one is homophilic, near zero is heterophilic. MDE asks the same thing about features: do linked nodes look alike? A large value means feature heterophily, and it describes the dataset rather than comparing methods. Both use the same set of length-two meta-paths from Hetero-squared-Net, and our first check is to reproduce their published numbers. Labels and features can disagree, so we report both, which answers the first reviewer concern.

28 Datasets (1): node classification, with their heterophily

Here are the eight node-classification datasets, four from HGB and four large ones from H2GB, now with their heterophily. All three columns use the heterophily form, so one means every linked pair differs. The superscripts show the source: H is Hetero-squared-Net's table, G is H2GB's, no mark means we computed it with H2GB's code, and a dot means still to compute. The main takeaway is that H-squared spans from about 0.17 to nearly one. IEEE-CIS-G shows why we prefer it: its MLH is 0.14, which looks homophilic, but after correcting for class imbalance its H-squared is 0.98. Two honest caveats. The numbers come from different meta-path sets, so the final table has to use a single set. And the star on Freebase is because most books are unlabelled and the reference code fills them at random; the labelled-only values are on the slide.

29 Datasets (2): link prediction and graph classification

Link prediction and graph classification. For links we have eight tasks. ACM, DBLP and IMDB are derived from HGB, then Freebase book-to-book, then PubMed and LastFM from the HGB link sets, and two large ones from H2GB, citations in ogbn-mag and friendships in H-Pokec. In every task we remove features that would leak the answer. Graph classification goes in the appendix:

four TU datasets. Their types come from atom or structure labels, so this shows coverage only. We make no heterophily claim there.

30 Evaluation metrics and protocol

Metrics and protocol. For nodes and graphs we use Macro-F1, where every class counts equally, and Micro-F1 for overall correctness. For links, ROC-AUC asks whether a true link scores above a hard negative, and MRR asks whether it ranks first. The protocol is one rule for everybody. Same data, same splits, same negatives. Tune on seed zero with 100 Bayesian trials. Retrain the best configuration on seeds zero to four. Report mean and standard deviation. Test with Welch's t-test, raw and adjusted p-values. Every baseline is rerun under this rule, HetSheaf included, and that is our answer to concern four.

31 Cost: how it grows, and what we measured

Now cost, which was concern three. Maps are applied, never stored as d-by-d matrices. Faces cost in proportion to I, the number of face incidences. The solve takes about ten steps, and memory doesn't grow with the number of steps. So the whole thing is linear in the edges and face incidences. For comparison, the submitted version had maps costing d cubed per edge, and HAN and SeHGNN build one graph per schema, which can be denser than the original. The chart shows peak GPU memory on ACM: 9.3 gigabytes in v1, 2.8 now. Be careful with the comparison though. The others come from the v1 paper's table, ours is a laptop GPU with d equal to 32, and fair timing has to wait for the server.

32 Ablation study: which part carries the gain?

The ablation study. Each row removes or replaces exactly one thing. Take away all faces: if the full model wins, path instances add information. Replace the face with a closing edge: a face is more than its endpoints. Give the head the path as a feature instead: structure beats instance features. Make the weight uniform: who is on the path matters. Set the edge maps to identity: the maps are what handle heterophily, though that row is still a proposal. Use free maps instead of composed ones: composition matters. One change per row, same paths, same budget, and we fix in advance what we'll write if the result goes against us. These are ablations, not rankings, because a well-tuned R-GCN can rival many heterogeneous GNNs.

33 Model analysis: when and why it works

Ablations tell us which part helps. The analyses tell us when and why. We plan six. Gain against heterophily, plotted over H-squared, for RQ1. Known-answer tasks:

synthetic graphs where the label depends on the intermediate node, to test the mechanism. Schema read-out: are the learned weights per schema sensible and stable? Path length, from two to six, for sensitivity. Scaling: memory and time against graph size and number of faces. And who gets weight, by intermediate type and by label match, for RQ2. Together they should tell us when it works, why, and what it costs.

34 What is new, and what is not

What is new, and what isn't. New: paths as faces of the graph's own sheaf; a face that reaches every node with no shortcut; heterophily and meta-paths in one objective; and ablations that are configurations of one model. Not new, and we say so in the paper: intermediate nodes in instance encoders, like MAGNN and SGAT; sheaves on typed graphs, which is HetSheaf; the solver, which has the form of personalised PageRank; and a special case of ours equals MAGNN's encoder. So the contribution isn't that paths are used. It's where the path acts: inside one sheaf, on the original graph.

35 Then and now: what changed since the submission

Then and now, in one table. The story went from three contributions to one idea. The face used to reach nodes only through the closing edge; now it reaches them directly. K Hodge steps became one objective and one solve. AMPSS with four budgets became WISP. Three node datasets became eight. HetSheaf quoted became every baseline rerun. ACM memory went from 9.3 to 2.8 gigabytes. The core idea stayed the same. What we replaced was its weakest link, the closing edge, and the weak evidence.

36 Limitations and next steps

Limitations and next steps. We state three limitations in the paper. Influence is symmetric in strength, and direction lives only in the maps. Propagation is linear, with the non-linearity in the head. And edge states carry no input of their own, so there are no edge features. The next steps, in order: the mini-batch design for the large H2GB graphs, the baseline plan including a HetSheaf adapter, then the server runs, and after that the analysis and ablation studies.

37 Conclusion

To wrap up. Existing methods handle heterophily on per-schema graphs and reduce each path to its endpoints. Our idea is to stay on the original graph, use maps to deal with heterophily, and make every path instance a face. One objective, one solve, and WISP samples and weights the paths. Next is RQ1 and

RQ2 on eight plus eight plus four datasets, with the ablations fixed in advance. The model is implemented and tested, but there are no accuracy numbers yet. Thank you, I'm happy to take questions.

38 References

Reference list. I won't present this slide; it's here for questions about a citation.

39 References (cont.)

Reference list. I won't present this slide; it's here for questions about a citation.

40 References (cont.)

Reference list. I won't present this slide; it's here for questions about a citation.

41 Backup slide

Backup slides start here. I'd only open these if someone asks.

42 Training: the loss for each downstream task

If someone asks about training. Node classification uses cross-entropy on the labelled training nodes. IMDB is multi-label, so we use one binary loss per genre and predict a genre when its logit is above zero. For links, the input is x-star joined with h, one negative per positive, and the decoder follows HGB. For graph classification we pool, with mean, max, add or attention, and add no new readout. The propagation is identical for every task. Only the head and the loss change.

43 Datasets (1): node classification, small to large

The full dataset table for node classification. Four from HGB, ACM, DBLP, IMDB and Freebase, and four from H2GB, IEEE-CIS-G, mag-year, ogbn-mag and H-Pokec. The slide shows nodes, node types, the target and how we train: full graph for the small ones, mini-batch for the large ones. HGB uses a 24, 6, 70 split and H2GB brings its own. Freebase memory hasn't been measured yet. We chose these to span the heterophily range.

44 Datasets (2): link prediction and graph classification

Same content as the earlier slide. Eight link tasks across HGB and H2GB graphs, with leaking features removed everywhere, and four TU datasets for graph classification. Their types come from labels, so we claim coverage only.

45 Heterophily metric (1): MLH, do linked nodes share a label?

This is MLH in detail. We take the length-two meta-paths from a target type back to the target type. Each one induces a graph on the target nodes. In our example, PAP links papers that share an author. Then we look at the label y of both ends. Near one, linked nodes share labels. Near zero, they differ. Look at the example: P1 and P9 are linked and differ. P1 and P4 are linked and match. H-squared then corrects this for class imbalance. The table at the bottom lists the meta-path sets we take from Hetero-squared-Net for each dataset, and other graphs use their get_metapaths rule.

46 Heterophily metric (2): MDE, do linked nodes look alike?

MDE is the feature version. For a target node u, x_u is its raw feature vector and N_u its neighbours in the meta-path graph. A large value means linked nodes have different features. It describes a dataset, and the scale depends on the features. The small matrix shows where each measure applies: MLH and H-squared work for nodes, for links on the labelled ends, and MLH has a one-hop version for graphs. MDE needs features, so not Freebase. Before anything else we reproduce Hetero-squared-Net's published values on DBLP, ACM and IMDB. And again: labels and features can disagree, and reporting both answers reviewer concern one.

47 Evaluation metrics

The metrics, each with its purpose. Macro-F1 averages F1 over classes, so rare classes count as much as common ones. Micro-F1 is overall correctness, and for single-label tasks it equals accuracy. ROC-AUC for links asks whether the true link scores above a 2-hop negative, with one negative per positive. MRR rewards ranking the true tail first among the candidates. Model selection uses validation Macro-F1 for nodes and graphs, and validation ROC-AUC for links, and that's fixed before we run anything.

48 Protocol: one rule for every method

The protocol again, with the reviewer concerns beside it. Everybody gets the same data, splits and negatives. We tune on seed zero with 100 Bayesian trials, retrain the best configuration on seeds zero to four, report mean and standard deviation, and test with Welch's t-test, which allows unequal variances. We compare against the best other method, and if a baseline is better than the one we named, we report that too. The table matches each concern to what we do: same budget for everyone, HetSheaf rerun on our data, and fixed splits with the

std taken over five training seeds. Each method keeps its own feature preprocessing, as in its code. Everything else is shared.

49 Hyperparameters: what is tuned, fixed and learned, per step

If someone asks how many hyperparameters there are, here they are step by step. In encoding we tune the dimension d and the encoder dropout. Edge maps have nothing tuned: two reflections and a width of 128 are fixed. WISP is fixed too, with maximum length four and a calibrated number of walks. The energy has only its normalisation. The solver has a tolerance of 1e-4 and a maximum of 100 steps, fixed, and tau is learned. The head tunes the classifier dropout. Training tunes the learning rate and weight decay. So the count goes from nine tuned plus four budgets in the submitted version to five tuned plus two now, and v1 also tuned the diffusion steps K and two step sizes.

50 Cost (1): how time and memory grow

Cost, step by step. N is the number of nodes, E the edges, F the faces, I the face incidences, k the number of reflections, which is two, and T about ten solver steps. The table gives time and memory for the WISP draw, the edge maps, the instance weights, the solve, and the backward pass, which is one extra solve. On the right, how others compare. v1 had d-by-d maps and memory that grew with K. HAN and SeHGNN build schema graphs that can be denser than E. MAGNN touches every instance of every schema. Ours is linear in edges and face incidences, scales with d and not d cubed, and memory doesn't grow with solver steps.

51 Cost (2): measured memory and solver steps

Measured numbers. The bar chart shows peak memory on ACM. The other methods come from the v1 paper's Table 11, and ours is on a laptop GPU with d equal to 32. On the right, our three HGB datasets. ACM has 103 thousand faces, 2.8 gigabytes, and about 9 solver steps forward and 8 backward. DBLP has 207 thousand faces, 5.0 gigabytes and 11 steps. IMDB has 38 thousand faces, 1.3 gigabytes and 10 steps. Notice that the number of solver steps stays between 9 and 11 while the state grows about four times. The epoch times came from a slow laptop session, so please don't read much into them. The version 1 time was 3.0 seconds on ACM, and fair timing will come from the server.

52 Ablation study: which part carries the gain?

Same ablation table, with a few extra rows. Besides the earlier ones: drop the node-to-face term, to show a face must reach nodes directly; and replace

orthogonal maps with diagonal ones, to show relation order matters. The weight row now has uniform, schema-level and endpoint-only variants. Same sampled paths, same parameter budget, same tuning. The shaded rows decide RQ2. Each row isolates one component, which is what the reviewers asked for, and the sentence we'll write for each outcome is fixed before we run.

53 Model analysis: when and why it works

The analysis plan with a bit more detail. Gain over the best baseline and over the no-faces variant, plotted against H-squared. Synthetic graphs where the label depends on the intermediate node. The share of weight per schema, and whether it's stable across seeds. Accuracy for maximum length two up to six. Memory, time and solver steps against graph size and number of faces. And a case study of instance weights, by intermediate type and label agreement. Ablations say which part helps. Analyses say when, why and at what cost.

54 What is new, and what is not

Same message as before, with slightly sharper wording. New: paths as faces of the graph's own sheaf, a face that reaches every node, heterophily on the original graph, and controls that are configurations of one model. Not new: intermediate nodes in MAGNN and SGAT, sheaves on typed graphs in HetSheaf, the solver as personalised PageRank, and the special case equal to MAGNN's encoder. The contribution is where the path acts, not that paths are used.

55 Then and now (1/2): story and model

The long version of then and now, first half. The story: from keeping path instances as 2-cells with three contributions, to heterophily on the original graph with paths as structure. The path cell: from a 2-cell closed by a synthetic closing edge, to a face above its path with no closing edge. Face to nodes: from only through the closing edge, proved zero otherwise, to direct, to every node. Propagation: from K Hodge steps to one energy and one solve. Selection: from AMPSS to WISP. Weights: from within selected schemas to every instance. Readout: from SheafCovPool to plain pooling. Memory on ACM is 0.30 of v1, on a laptop GPU with d equal to 32. Two components were dropped, the core idea was kept, and its weakest link was removed.

56 Then and now (2/2): evaluation

Second half, the evaluation. Node classification goes from three datasets, one strongly heterophilic, to eight including four large H2GB graphs. Link prediction goes from four tasks, one with a single node type, to eight tasks with real types

and leak-free features. Graph classification now follows HetSheaf's protocol, and says plainly that types come from labels. Heterophily evidence goes from MLH only to MLH, H-squared and MDE, with gains plotted against H-squared. Baselines are all rerun with 100 Bayesian trials each. Statistics go from mean and std to five seeds and Welch's test. Ablations become pre-registered controls. Each row answers one reviewer concern, and none of it has been run yet.

57 Limitations and next steps

Same limitations and next steps as in the main part, in case someone wants to go back to it. Three limitations: symmetric strength of influence, linear propagation, and no edge features. Then the next steps in order: the mini-batch design, the baseline plan, server runs, and the analysis.

58 Conclusion

Same conclusion as the main part. The idea: stay on the original graph, with each path instance as a face reaching all its nodes. One energy, one solve, and WISP samples and weights the paths. The evidence plan: RQ1 and RQ2 on eight plus eight plus four datasets, with controls fixed in advance. The model is implemented and checked, and we have no accuracy numbers yet. Thank you.

59 References

Reference list. I won't present this slide; it's here for questions about a citation.

60 References (cont.)

Reference list. I won't present this slide; it's here for questions about a citation.

61 Appendix: terms used in this talk

This glossary is for questions. Heterogeneous graph: nodes and edges have types. Heterophily: linked nodes tend to have different labels or features. Meta-path schema: a type pattern, like paper-author-paper. Path instance: one concrete walk that follows it, like P1, A7, P9. Restriction map F: a learned map from a node into the space of an edge. Face: a cell placed above one sampled path instance, with no closing edge. Energy: the sum of disagreements between touching cells, after the maps. WISP: walk-sampled, instance-weighted sheaf paths, which is the sampler plus the weights. And tau: the learned strength of agreement in the solve.